

AI Infra · 推理服务与缓存

Paged KV、CUDA Graph 与 Serving：先确定目标函数，再选择数据结构

Paged KVPrefix CacheServingFlashAttention

岗位能力画像

岗位画像：考察对请求生命周期、KV 物理布局、前缀命中、图捕获限制和在线/离线目标差异的理解。

高频深挖：题目背后的判断

Paged Attention 到底解决什么？

它解决动态 KV 的非连续管理、分配和调度，不等于自动跳过 full attention 计算。

块表是内存管理与寻址结构。

前缀缓存如何避免错命中？

块 key 需串接前缀身份、当前 token、位置/模型等必要上下文；从头连续匹配。

第一个 miss 后只计算后缀。

Graph 什么时候值得 capture？

固定拓扑、shape、地址/生命周期且 launch 开销显著时；host sync/动态分配是风险。

Graph 解决提交空洞，不替代算法优化。

在线与离线优化有什么不同？

在线优先 TTFT/TPOT、尾延迟和可抢占；离线优先总吞吐、利用率和失败恢复。

先定义服务目标函数。

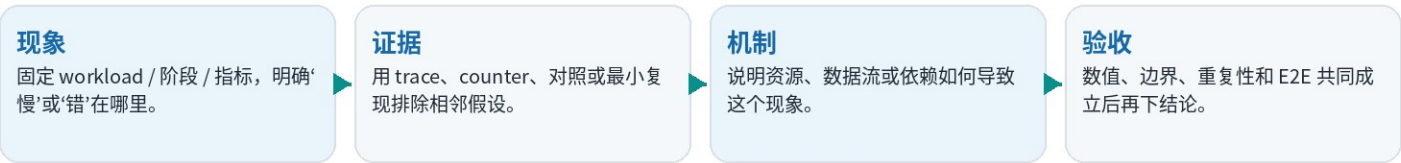
项目叙事的通用深挖链



答题提醒

不要从‘用了什么库/指令’开始；先固定工作负载、指标和约束，再说明数据流/资源/依赖，最后给出正确性、性能和反例边界。

把一句结论变成可复验的事实



一题答透：从 FlashAttention 到 Paged KV：两种‘不落 HBM’的思考

问题 “怎样解释 online softmax 和 KV block table 的共同点与边界？”

Attention

传统实现会 materialize score/probability 中间矩阵；分块 online softmax 维护 running max、normalizer 与 accumulator。

KV layout

token 序列按定长块映射到物理 KV blocks；block table 把逻辑序列与物理位置解耦。

命中判断

前缀 key 链式构造，从头连续命中；避免交换 token 后仍误命中。

验收

分别测命中率、显存碎片、TTFT/TPOT、吞吐与失败恢复，不混为一个‘加速比’。

把推理链画出来



常见误区

× 把 paging 说成 sparse attention。

× 前缀 hash 没有位置或上下文，造成错误复用。

× 没有区分 online 与 offline 的目标函数。

术语速懂 | 面试怎么答

术语速懂

Paged KV

用 block table 将逻辑 token 块映射到物理 KV 块，解决动态分配、回收和碎片，不等于 sparse attention。

Prefix Cache key

前缀必须从头连续命中；key 要编码必要上下文，防止 token 重排或上下文变化导致错误复用。

面试中怎么答

- ① 先区分 KV 寻址、前缀复用与 attention 计算分别解决什么。
- ② 再讲 block table、refcount、ready / lease 与第一个 miss 后的处理。
- ③ 分别验证碎片、TTFT/TPOT、取消/恢复、碰撞和回收。

白板 / 手撕：白板题：请求调度与缓存指标

题面 设计 prefill/decode 队列、块表与 prefix cache；给出优先级、回收和监控指标。

C++17 · continuous prefix lookup

核心实现

```
01 using Key = uint64_t; // hash(parent, block, pos, context)
02 struct Entry { bool ready; uint32_t refcount; };
03 size_t longest_ready_prefix(const vector<Key>& chain,
04     const unordered_map<Key, Entry>& cache) {
05     size_t hit = 0;
06     for (Key key : chain) {
07         auto it = cache.find(key);
08         if (it == cache.end() || !it->second.ready) break;
09         ++hit; // stop at the first miss
10     }
11     return hit;
12 }
```

写代码时怎么讲

1. 请求状态：admit → prefill → decode → finish/cancel；每次状态变化更新 block ownership。
2. 队列策略说明 prefill/decode 取舍、预算与可抢占；不只说 FIFO。
3. 块表包含 logical block、physical block、refcount、ready/lease 和 eviction。
4. 指标至少有 TTFT、TPOT、queue、hit、fragmentation、transfer、retry/failure。

实现重点

- 必须从头连续命中，不能跳过 miss 到后面复用。
- 实际消费前需原子增加 refcount 或取得 lease。
- key 必须包含位置和必要上下文，防止错误复用。

边界、测试与追问

最少要说明的验证

- 必须从头连续命中，不能跳过 miss 到后面复用。
- 实际消费前需原子增加 refcount 或取得 lease。
- key 必须包含位置和必要上下文，防止错误复用。

面试继续追问

- full attention 时 paging 还有什么价值？
- block key 如何支持共享前缀又避免碰撞？
- Graph replay 为什么依赖地址和生命周期？